

Cloud Foundations Virtual Bootcamp

AWS Sales Data Pipeline: A Cloud-Based Data Engineering Solution for Automated Sales Data Processing and Business Insights

Prepared by: Faith Ogundusi

Email Address: ogundusifaith.1@gmail.com

August 2026

TABLE OF CONTENTS

1. Introduction
2. Business Problem
3. Project Objectives
4. Proposed Solution
5. Solution Architecture
6. AWS Services Used
7. Dataset
8. Data Pipeline Implementation
9. Pipeline Execution Results
10. Amazon RDS Data Storage
11. Amazon SNS Notifications
12. Sales KPI Dashboard
13. GitHub Deployment and Documentation
14. Benefits to Customers and Business Users
15. Challenges Encountered
16. Lessons Learned
17. Future Enhancements
18. Conclusion
19. Project Links
20. Author

INTRODUCTION

The AWS Sales Data Pipeline is a cloud-based data engineering project designed to demonstrate how raw sales data can be processed, validated, stored, monitored, and transformed into useful business insights using Amazon Web Services (AWS). The project uses a transactional online retail dataset obtained through Kaggle and implements a simple cloud-based data pipeline using Amazon S3, AWS Lambda, Amazon RDS, Amazon SNS, and AWS IAM.

The pipeline begins with raw sales data stored in Amazon S3. AWS Lambda processes and validates the records before separating them into valid and rejected datasets. Accepted records are stored in Amazon RDS, while rejected records are retained separately for review. Amazon SNS is used to provide pipeline completion and alert notifications. The processed sales information is also presented through a Sales KPI Dashboard deployed using GitHub Pages.

The project demonstrates practical knowledge of cloud data engineering, serverless computing, data validation, database storage, notifications, data visualization, and technical documentation.

BUSINESS PROBLEM

Businesses generate large volumes of transactional sales data. Before such data can be used for reporting and decision-making, it needs to be properly processed and validated. Raw sales datasets may contain incomplete or invalid records, including missing required fields, zero or negative quantities, and zero or negative unit prices.

When data processing is performed manually, identifying invalid records, maintaining organized datasets, and preparing information for analysis can become time-consuming and difficult to scale.

The business problem addressed by this project is therefore: How can raw sales data be processed automatically, invalid records be identified and separated, accepted records be stored in a structured database, and useful sales information be made available for business analysis?

The proposed AWS solution addresses this problem through an automated cloud-based processing workflow.

PROJECT OBJECTIVES

The main objectives of the project were to:

- Build a simple cloud-based sales data pipeline.
- Store raw sales data using Amazon S3.
- Process and validate sales records using AWS Lambda.
- Separate valid records from rejected records.
- Store rejected records separately for review.
- Store accepted records in an Amazon RDS MySQL database.
- Use Amazon SNS for pipeline completion and alert notifications.
- Use Amazon EventBridge to schedule automated pipeline execution.
- Develop a Sales KPI Dashboard from the processed data.
- Deploy the project documentation and dashboard through GitHub.
- Demonstrate practical understanding of AWS services and cloud architecture.

PROPOSED SOLUTION

The proposed solution is a cloud-based sales data pipeline designed to automate the ingestion, validation, processing, storage, monitoring, and notification of sales data using AWS managed services.

The pipeline begins with the Online Retail sales dataset obtained from Kaggle. The CSV file is uploaded to an Amazon S3 raw data bucket, which serves as the initial storage location for incoming sales data.

The pipeline uses Amazon EventBridge to automate execution of the AWS Lambda processing function. An enabled scheduled rule named sales-pipeline-monthly-trigger triggers the Lambda function at midnight UTC on the first day of every month, reducing the need for manual execution.

AWS Lambda performs the main data processing and validation activities. The function parses the sales records and applies predefined data-quality and business validation rules. These include checking for missing required values, zero or negative quantities, and zero or negative unit prices.

Records that pass the validation process are classified as valid sales records. The validated records are stored in the appropriate Amazon S3 location and loaded into an Amazon RDS MySQL database for structured storage, querying, and downstream analysis.

Records that fail the validation rules are classified as rejected records. Rather than deleting these records, the pipeline stores them separately in Amazon S3 together with relevant error information. This provides traceability and allows data-quality issues to be reviewed and addressed.

Amazon SNS is incorporated into the solution to provide operational notifications. Amazon SNS is used to provide operational notifications, including pipeline completion notifications and alerts associated with processing outcomes.

Amazon CloudWatch provides monitoring and logging for the Lambda processing environment. Lambda execution logs and errors can be reviewed through CloudWatch to support troubleshooting and operational monitoring.

The processed and validated sales data is used to produce a Sales KPI Dashboard, which presents business-focused information such as revenue performance, order trends, product performance, customer information, country-level sales, and sales

trends over time.

This solution uses managed AWS services to create an automated and scalable data-processing workflow while maintaining separation between raw data, validated data, rejected data, structured database storage, monitoring, and business visualization.

SOLUTION ARCHITECTURE

The solution architecture consists of several interconnected components.

- DATA SOURCE

The project uses an online retail sales dataset obtained through Kaggle. The dataset is provided as a CSV file and serves as the raw input to the pipeline.

- AMAZON S3

Amazon S3 acts as the main object storage layer. The project organizes the processed data into:

- Data/raw/
- Data/valid/
- Data/rejected/

The raw folder contains the original sales data. The valid folder contains records that passed the validation rules. The rejected folder contains records that failed validation.

- AWS LAMBDA

AWS Lambda is responsible for serverless data processing and validation. The Lambda function reads the sales data, applies validation rules, and separates the records into valid and rejected outputs. This eliminates the need to maintain a dedicated server for the processing workload.

- AMAZON RDS

Amazon RDS provides relational database storage for the accepted sales records. The project uses MySQL as the database engine. This allows accepted sales information to be stored in a structured format suitable for querying and analysis.

- AMAZON SNS

Amazon SNS provides notification capabilities. The project uses SNS for pipeline completion notifications and alert notifications. This provides operational awareness when pipeline processing occurs or requires attention.

- AWS IAM

AWS IAM is used to manage access and permissions for AWS resources. IAM roles and permissions allow the required AWS services to interact while maintaining controlled access to resources.

- AMAZON EVENTBRIDGE

Amazon EventBridge is used as the scheduling component of the pipeline. The enabled rule, sales-pipeline-monthly-trigger, uses the schedule expression `cron(0 0 1 * ? *)`, which runs at midnight UTC on the first day of every month. The rule targets the AWS Lambda processing function, enabling scheduled execution of the sales pipeline without requiring manual initiation.

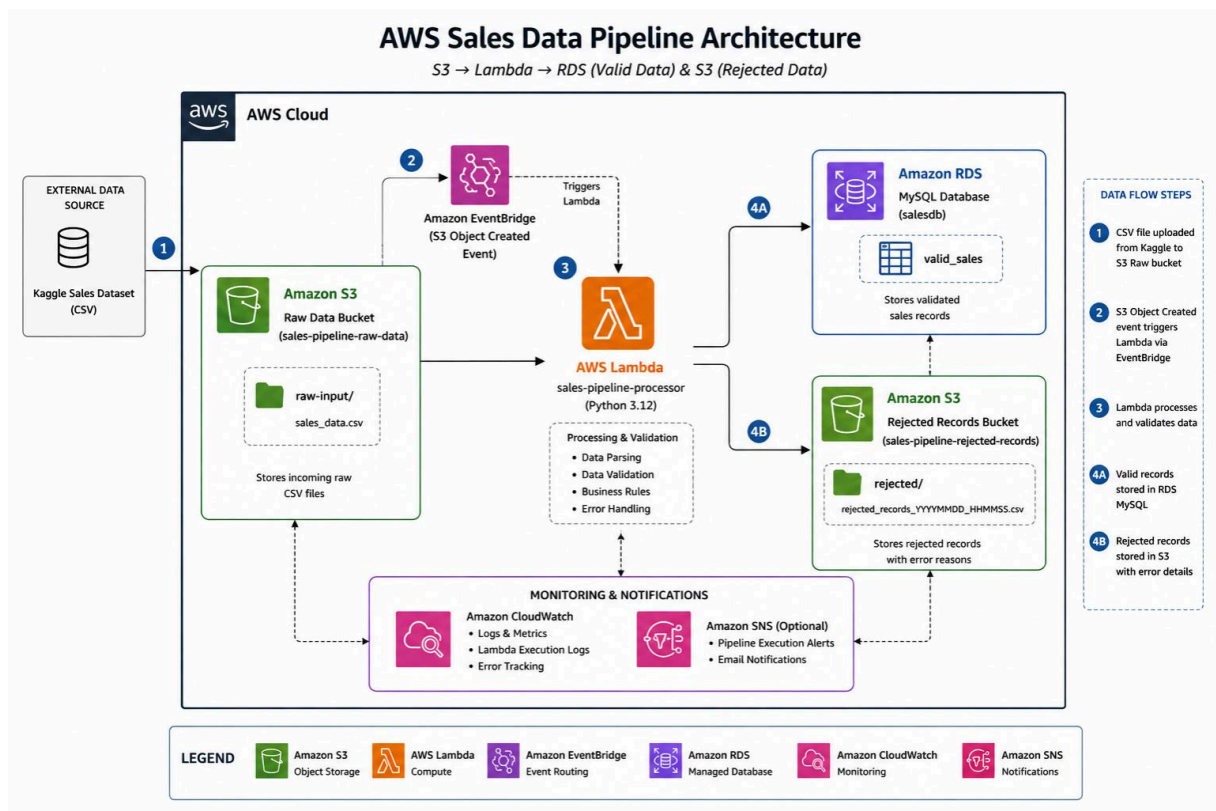


Figure 1: AWS Sales Data Pipeline Architecture

The architecture diagram illustrates the movement of sales data from the source dataset through Amazon S3 and AWS Lambda processing, into valid and rejected outputs, Amazon RDS, Amazon SNS notifications, and the Sales KPI Dashboard.

AWS SERVICES USED

AWS SERVICE	ROLE IN THE PROJECT
Amazon S3	Stores raw, valid, and rejected sales data
Amazon EventBridge	Schedules automated execution of the sales pipeline
AWS Lambda	Validates and processes sales records
Amazon RDS	Stores accepted records in MySQL
Amazon SNS	Sends completion and alert notifications
AWS IAM	Manages access and permissions
Amazon CloudWatch	Monitors Lambda execution logs and errors

Each service was selected based on a specific requirement within the pipeline rather than adding services without a functional purpose.

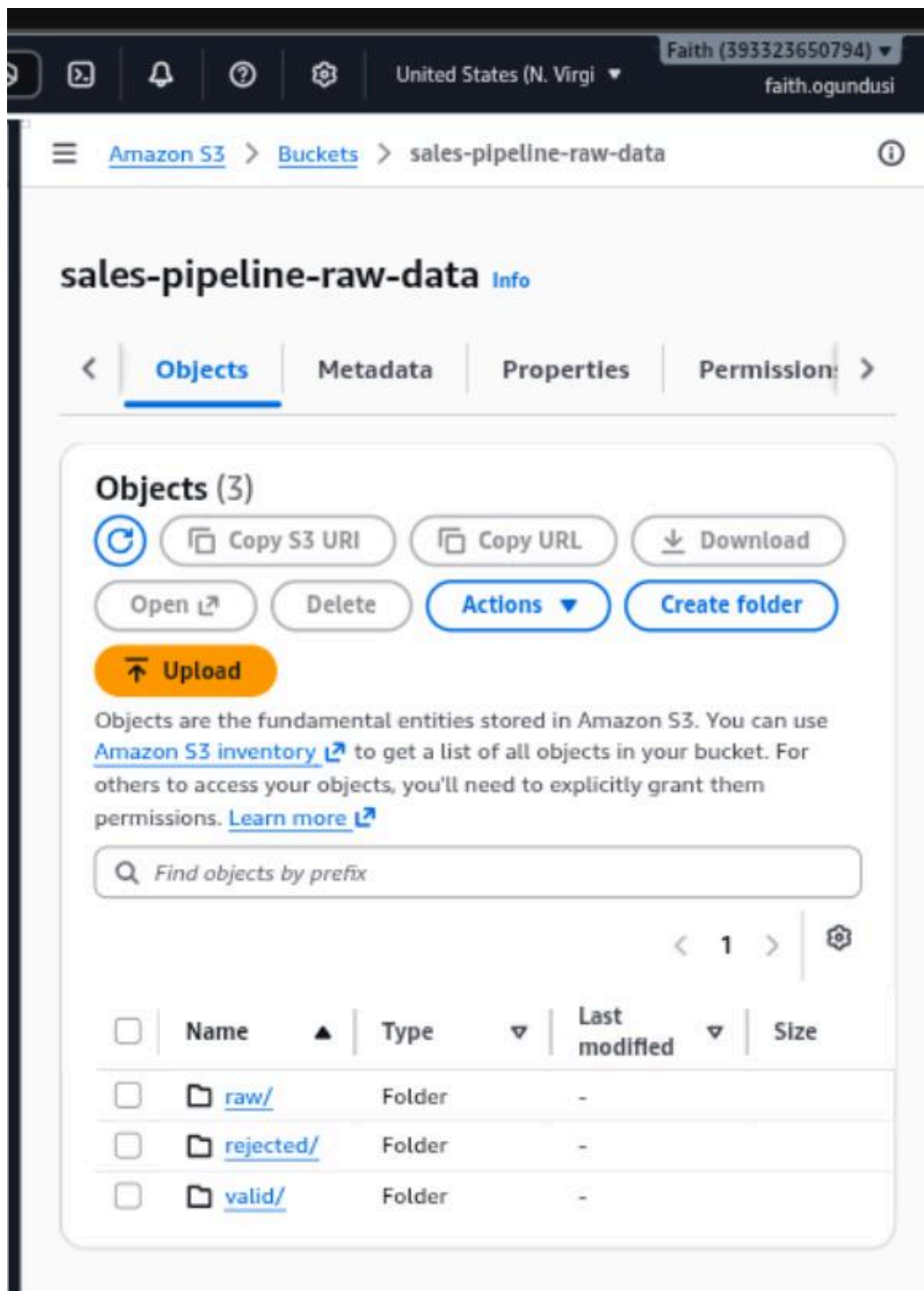


Figure 2: Amazon S3 storage structure

The screenshot shows the storage environment used for the project and the organization of raw, valid, and rejected data.

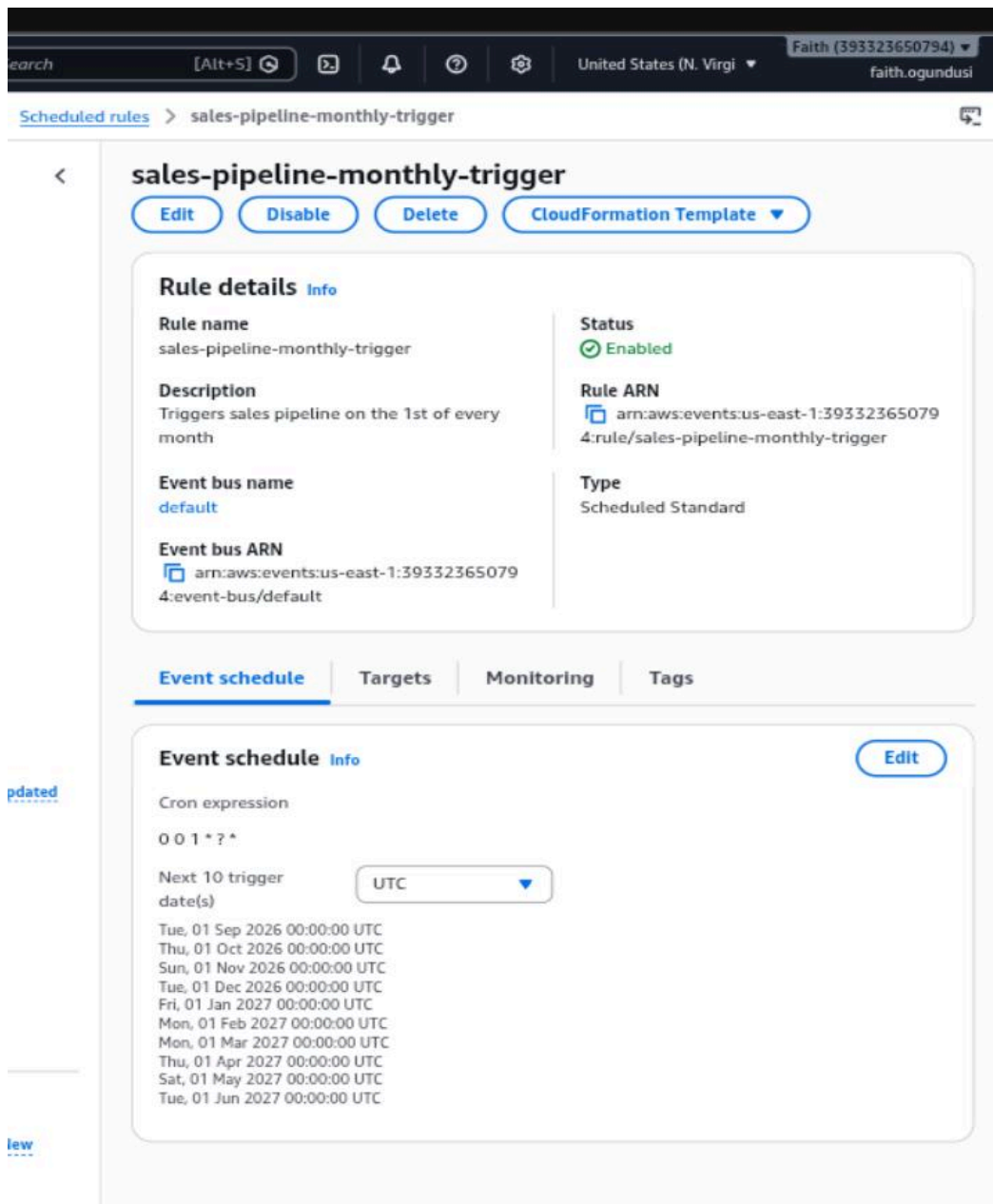


Figure 3: Amazon EventBridge Scheduled Trigger

The EventBridge rule `sales-pipeline-monthly-trigger` is enabled and configured to execute at midnight UTC on the first day of every month. The rule targets the AWS Lambda processing function.

aws

Search [Alt+S]

United States (N. Virgi)

Faith (393323650794)

faith.ogundusi

Lambda > Functions > sales-pipeline-processor

sales-pipeline-processor

Throttle Copy ARN Actions

Function overview Info

Diagram Template

sales-pipeline-processor

Layers (1)

EventBridge

S3

+ Add trigger

+ Add destination

Function ARN

arn:aws:lambda:us-east-1:393323650794:function:sales-pipeline-processor

Description

Last modified

6 hours ago

Export to Infrastructure Composer

Download

Code Test Monitor Configuration Aliases Versions

Code source Info

Open in Visual Studio Code

Update

lambda_function.py

SALES-PIPELINE-PROCESSOR

lambda_function.py

DEPLOY

Current

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+H)

TEST EVENTS [NONE SELECTED]

+ Create new test event

ENVIRONMENT VARIABLES

lambda_function.py

```

1 import json
2 import boto3
3 import csv
4 import io
5 import logging
6 import pymysql
7 from datetime import datetime
8
9 logger = logging.getLogger()
10 logger.setLevel(logging.INFO)
11
12 RDS_HOST = "sales-pipeline-db.cgfsy0yiksgv.us-east-1.rds
13 RDS_USER = "admin"
14 RDS_PASSWORD = "Faithiee12"
15 RDS_DB = "salesdb"
16 REJECTED_BUCKET = "sales-pipeline-raw-data"
17 REJECTED_PREFIX = "rejected/"
18
19 s3 = boto3.client("s3")
20
21
22 def ensure_table_exists(conn):
23     sql = (
24         "CREATE TABLE IF NOT EXISTS sales_transactions ( "
25         "id INT AUTO_INCREMENT PRIMARY KEY, "
26         "invoice_no VARCHAR(20), "
27         "stock_code VARCHAR(20), "
28         "description VARCHAR(255), "
29         "quantity INT, "
30         "invoice_date DATETIME, "
31         "unit_price DECIMAL(10,2), "

```

Code properties Info

Package size

2.2 kB

SHA256 hash

J09uePSBxfodpWUIOzahTqzkkBhfo8hZndPGDFXs9lc=

Code storage mode

Copy (default)

S3 location

arn:aws:s3::prod-04-2014-tasks/snapshots/393323650794/sales-pipeline-processor-62aadcd1-f0af-4c54-bebe-8aee23e83875

Last modified

6 hours ago

Runtime settings Info

Runtime

Python 3.12

Handler

Info

lambda_function.lambda_handler

Architecture

Info

x86_64

Edit

Edit runtime management configuration

Figure 4: AWS Lambda processing function

AWS Lambda performs the serverless validation and processing stage of the pipeline.

DATASET

The project uses the Online Retail Dataset obtained through Kaggle. The dataset contains transactional sales records from a UK-based online retailer.

The main fields include:

- Invoice Number
- Stock Code
- Product Description
- Quantity
- Invoice Date
- Unit Price
- Customer ID
- Country

The dataset contains transactional information that can be used to calculate revenue and generate sales performance metrics.

Revenue was calculated using: $\text{Revenue} = \text{Quantity} \times \text{UnitPrice}$

DATA PIPELINE IMPLEMENTATION

The AWS Sales Data Pipeline was implemented as an automated workflow using Amazon S3, Amazon EventBridge, AWS Lambda, Amazon RDS, Amazon SNS, and Amazon CloudWatch.

1. **DATA INGESTION:** The Online Retail sales dataset obtained from Kaggle was provided as a CSV file and uploaded to the Amazon S3 raw data bucket. Amazon S3 serves as the initial storage layer for incoming sales data. The raw data is kept separately from the processed outputs to preserve the original dataset and maintain a clear data-processing workflow.
2. **SCHEDULED AUTOMATION:** Amazon EventBridge was configured with a scheduled rule named sales-pipeline-monthly-trigger. The rule is enabled and uses the schedule expression `*cron(0 0 1 * ? )`. This schedule runs at midnight UTC on the first day of every month and targets the AWS Lambda processing function, enabling the sales pipeline to run automatically without requiring manual initiation.
3. **DATA PROCESSING:** AWS Lambda performs the main data processing and validation activities. The Lambda function is implemented using Python 3.12. The function reads the sales data, parses the records, applies the defined validation rules, and separates the records into valid and rejected outputs. The processing stage also handles errors and prepares the appropriate outputs for downstream storage.
4. **VALID RECORD PROCESSING:** Records that satisfy the defined validation criteria are classified as valid records. The valid records are stored in Amazon S3 and subsequently loaded into the Amazon RDS MySQL database. Amazon RDS provides structured relational storage that allows the accepted sales data to be queried and used for downstream analysis.

5. **REJECTED RECORD PROCESSING:** Records that fail the validation criteria are classified as rejected records. Rejected records are not deleted. Instead, they are stored separately in Amazon S3 together with information about the validation or processing issue. This approach improves traceability and allows rejected data to be reviewed for possible correction or further investigation.
6. **DATA VALIDATION:** The pipeline applies basic data-quality and business validation rules to determine whether individual records should be accepted. records may be rejected when they contain:
 - Missing required values.
 - Zero or negative quantity values.
 - Zero or negative unit price values.

These validation checks help prevent clearly invalid transactional records from being included in the accepted dataset.

7. **DATABASE STORAGE:** After validation, accepted sales records are stored in the Amazon RDS MySQL database. The database provides structured storage for the processed sales information and creates a suitable foundation for SQL queries, reporting, and business analysis.
8. **NOTIFICATIONS:** Amazon SNS is used to provide operational notifications from the pipeline. SNS is used for pipeline completion notifications and alert notifications associated with processing or rejected records. This provides visibility into pipeline activity and helps indicate when processing has completed or when attention may be required.
9. **MONITORING AND LOGGING:** Amazon CloudWatch is used to support monitoring of the AWS Lambda processing environment. Lambda execution logs and error information can be reviewed through CloudWatch to assist with troubleshooting, monitoring, and understanding pipeline execution behaviour.

10. BUSINESS VISUALIZATION: The accepted sales data is used to generate a Sales KPI Dashboard. The dashboard transforms the processed transactional data into business-focused information, including revenue KPIs, order trends, product performance, customer information, country-level sales, and sales trends over time. The dashboard is deployed through GitHub Pages to provide a publicly accessible presentation of the project results.

PIPELINE EXECUTION RESULTS

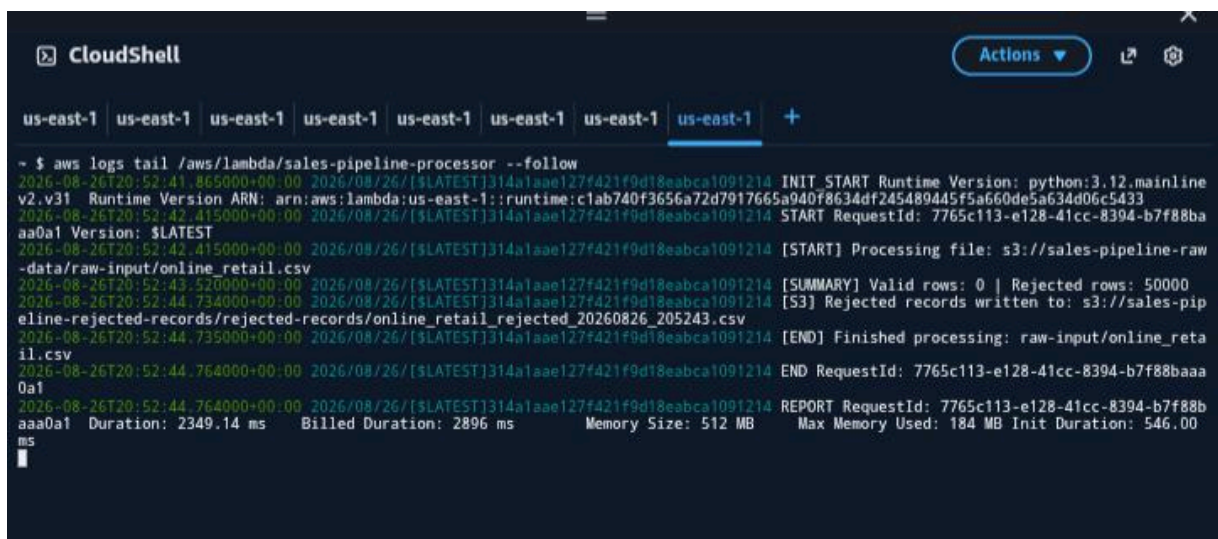
The pipeline was tested using 50,000 sales records.

The execution produced the following results:

Result	Records	Percentage
Valid / Accepted	30,315	60.6%
Rejected	19,685	39.4%
Total	50,000	100%

A total of 30,315 records, representing 60.6% of the processed dataset, passed the defined validation rules and were classified as valid. A total of 19,685 records, representing 39.4% of the processed dataset, failed one or more validation checks and were classified as rejected. The rejected records were retained separately in Amazon S3 with error details instead of being deleted. This provides a clear audit trail and makes it possible to investigate data-quality issues.

The execution results demonstrate that the pipeline can process transactional sales data, apply defined validation rules, separate accepted and rejected records, and prepare the resulting data for downstream storage and analysis.



```
CloudShell
us-east-1 us-east-1 us-east-1 us-east-1 us-east-1 us-east-1 us-east-1 us-east-1 +
~ $ aws logs tail /aws/lambda/sales-pipeline-processor --follow
2026-08-26T20:52:41.865000+00:00 2026/08/26/[$LATEST]314a1aae127f421f9d18eabca1091214 INIT_START Runtime Version: python:3.12.mainline
v2.v31 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:clab740f3656a72d7917665a940f8634df245480445f5a660de5a634d06c5423
2026-08-26T20:52:42.415000+00:00 2026/08/26/[$LATEST]314a1aae127f421f9d18eabca1091214 START RequestId: 7765c113-e128-41cc-8394-b7f88ba
aa0a1 Version: $LATEST
2026-08-26T20:52:42.415000+00:00 2026/08/26/[$LATEST]314a1aae127f421f9d18eabca1091214 [START] Processing file: s3://sales-pipeline-raw
-data/raw-input/online_retail.csv
2026-08-26T20:52:43.520000+00:00 2026/08/26/[$LATEST]314a1aae127f421f9d18eabca1091214 [SUMMARY] Valid rows: 0 | Rejected rows: 50000
2026-08-26T20:52:44.734000+00:00 2026/08/26/[$LATEST]314a1aae127f421f9d18eabca1091214 [S3] Rejected records written to: s3://sales-pip
eline-rejected-records/rejected-records/online_retail_rejected_20260826_205243.csv
2026-08-26T20:52:44.735000+00:00 2026/08/26/[$LATEST]314a1aae127f421f9d18eabca1091214 [END] Finished processing: raw-input/online_reta
il.csv
2026-08-26T20:52:44.764000+00:00 2026/08/26/[$LATEST]314a1aae127f421f9d18eabca1091214 END RequestId: 7765c113-e128-41cc-8394-b7f88baa
aa0a1 Duration: 2349.14 ms Billed Duration: 2896 ms Memory Size: 512 MB Max Memory Used: 184 MB Init Duration: 546.00
ms
```

Figure 5: Pipeline Processing Execution Evidence

The screenshot provides evidence of the Lambda pipeline execution and the processing of the sales dataset.

t+S]		
United States (N. Virgi		
Faith (393323650794)		
faith.ogundusi		
+     <		
Console		
X		
Pipeline Execution Summary — Confirmed from AWS CloudWatch & S3		
Metric	Value	Evidence Source
Total Raw Records Processed	50,000	Lambda log: Valid(30315) + Rejected(19685)
Valid / Accepted Records	30,315 (60.6%)	Lambda log: Valid: 30315
Rejected Records	19,685 (39.4%)	Lambda log: Rejected: 19685
Accepted Data File	valid/valid_sales.csv (3.6 MB)	S3: sales-pipeline-raw-data
Rejected Data File	rejected/rejected_20260827_132447.csv	S3: sales-pipeline-raw-data
RDS Destination	salesdb.sales_transactions — 30,315 rows	Lambda log: Inserted 30315 records into RDS
Pipeline Run Timestamp	2026-08-27 13:24 UTC	Lambda CloudWatch execution log
Lambda Runtime	Python 3.12 256 MB ~46s duration	Lambda REPORT log
SNS Alert Topic	sales-pipeline-alerts	SNS ListTopics
Dataset Columns	InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country	UCI Online Retail schema
Revenue Formula	Quantity × UnitPrice	Standard Online Retail calculation

Figure 6: Pipeline Execution Summary

The screenshot provides supporting evidence of the number of valid and rejected records produced during pipeline processing.

AMAZON RDS DATA STORAGE

Accepted sales records were stored in an Amazon RDS MySQL database. The purpose of Amazon RDS in the architecture is to provide structured relational storage for the accepted sales data. Using a relational database provides a foundation for querying and performing structured analysis on the processed records.

[Alt+S]

United States (N. Virgi)
Faith (393323650794)
faith.ogundusi

> sales-pipeline-db

sales-pipeline-db

Summary

DB Identifier
sales-pipeline-db

CPU
3.72%

Role
Instance

Current activity
0 Connections

Recommendations

Status
Available

Class
db.t3.micro

Engine
MySQL Community

Region & AZ
us-east-1f

Connectivity & security
Monitoring
Logs & events
Configure

Connect using

Code snippets
Use when connecting through SDK, APIs, or third-party tools including agents.

CloudShell
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.

Endpoints
Use when connecting through any IDE interface.

Database name
salesdb

Master username
admin

Internet access gateway
Disabled

Port
3306

Endpoint type
Instance endpoint

Additional configurations

Connectivity & security

Endpoint & port

Networking

Security

24

Figure 7: Amazon RDS database environment

The screenshot provides evidence of the relational database component used to store accepted sales records.

AMAZON SNS NOTIFICATIONS

Amazon SNS was incorporated into the pipeline to provide operational notifications.

The project uses SNS for:

- Pipeline completion notifications.
- Alert notifications.

The notification component improves visibility into the pipeline and can help users identify when processing has completed or when attention may be required.

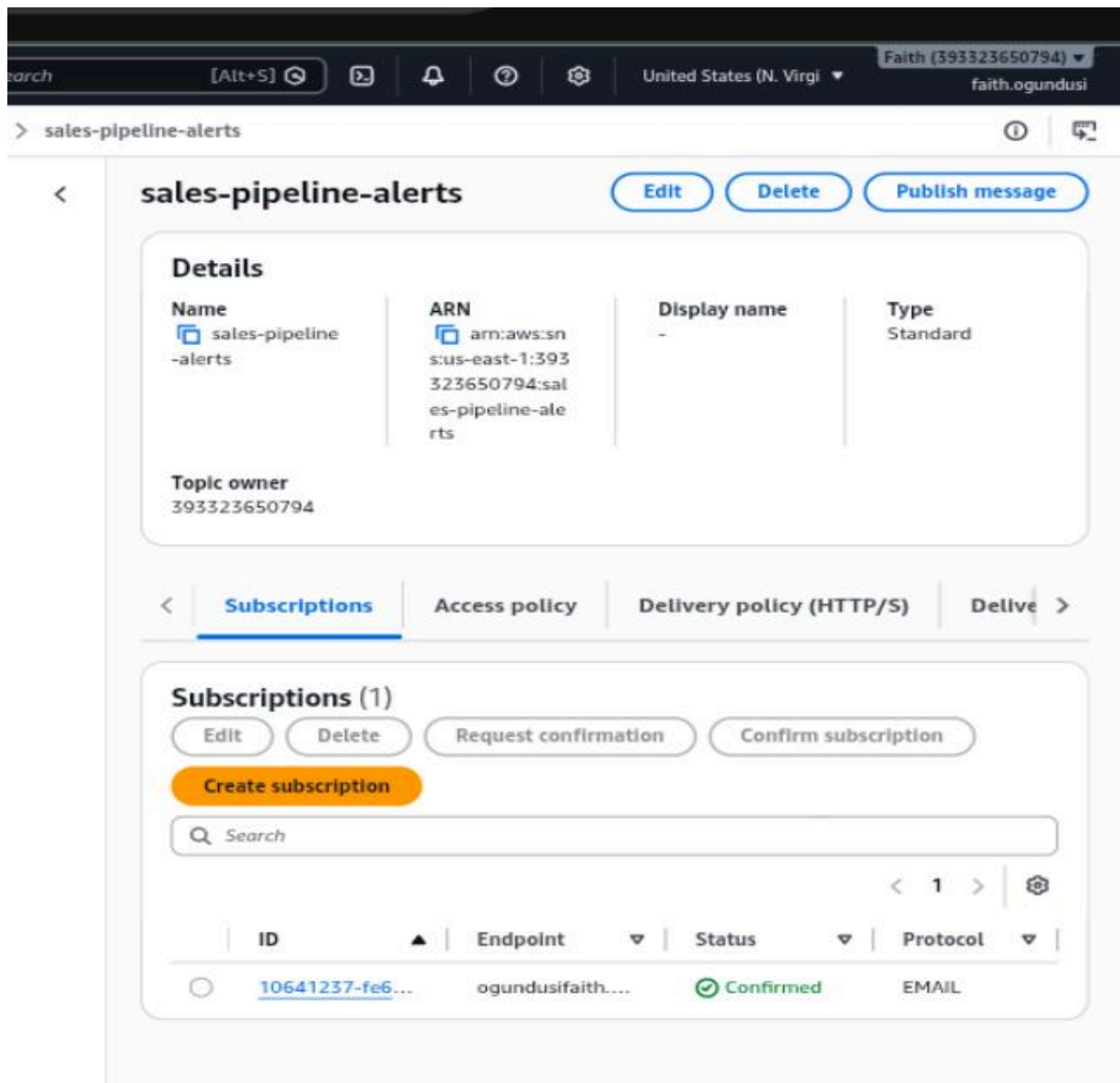


Figure 8: Amazon SNS notification configuration

The screenshot provides evidence of the notification component used in the pipeline.

SALES KPI DASHBOARD

A Sales KPI Dashboard was developed using the accepted pipeline data. The dashboard was created to transform processed transactional data into business-focused information that can be easily understood.

The dashboard includes:

- Revenue KPIs
- Order trends
- Top products by revenue
- Top products by quantity
- Revenue by country
- Top customers
- Sales trends over time

The dashboard is implemented as: Visualizations/dashboard.html

The dashboard has also been deployed through GitHub Pages.

Live Dashboard: <https://faith-ogundusi.github.io/aws-sales-data-pipeline/>

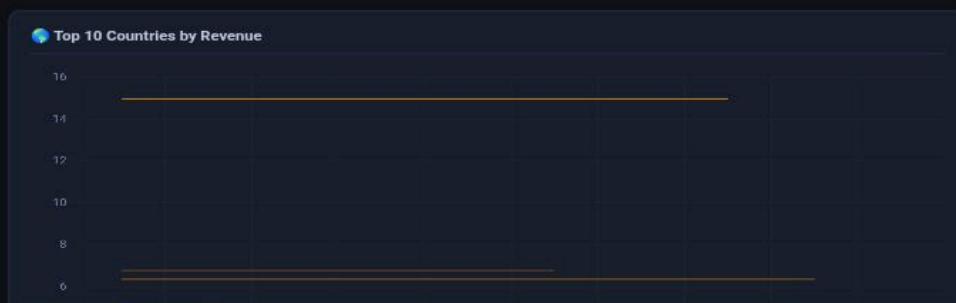
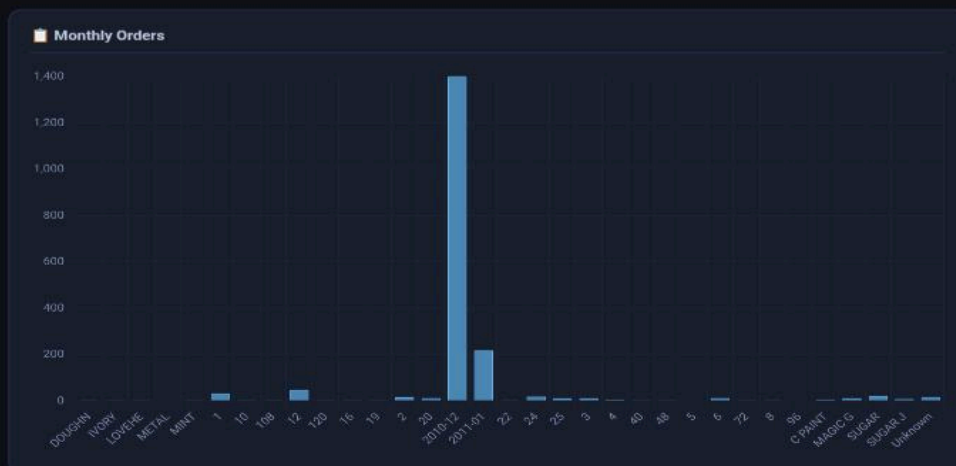
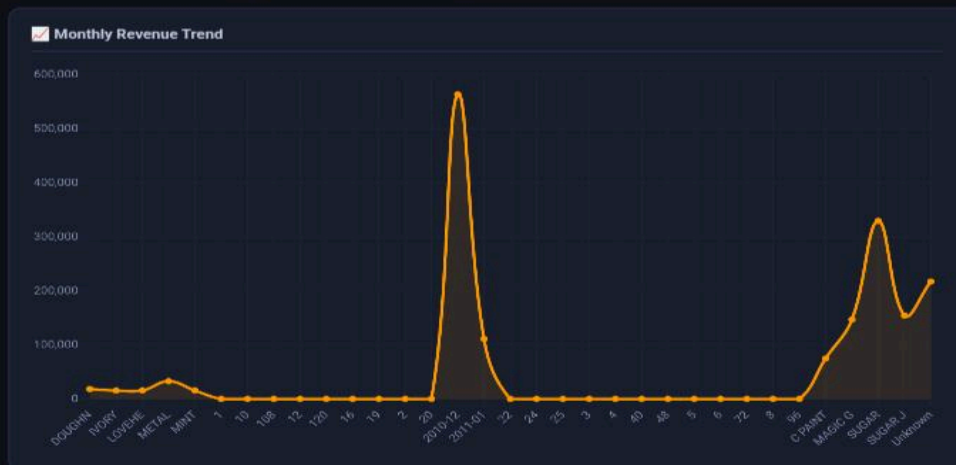


Figure 9: Sales KPI Dashboard

The dashboard provides a visual summary of sales performance using the accepted sales records.

GITHUB DEPLOYMENT AND DOCUMENTATION

GitHub was used as the project repository and documentation platform. The repository contains the project data organization, dashboard, architecture documentation, README, and supporting project files.

Repository: <https://github.com/faith-ogundusi/aws-sales-data-pipeline>

The GitHub Pages deployment provides a publicly accessible version of the sales KPI Dashboard.

Live deployment: <https://faith-ogundusi.github.io/aws-sales-data-pipeline/>

REPOSITORY STRUCTURE

The project repository is organized as follows:

Aws-sales-data-pipeline/

|

|— data/

| |— raw/

| |— valid/

| |— rejected/

|

|— visualizations/

| |— dashboard.html

|

|— architecture/

| |— architecture.png

|

|— LICENSE

|— README.md

BENEFITS TO THE BUSINESS AND BUSINESS USERS

The solution provides several business and technical benefits.

1. **IMPROVED DATA QUALITY:** The validation stage helps prevent clearly invalid records from entering the accepted dataset.
2. **REDUCED MANUAL PROCESSING:** Automated processing reduces the need to manually inspect and separate large numbers of sales records.
3. **BETTER DATA ORGANIZATION:** Amazon S3 provides a clear separation between raw, valid, and rejected datasets.
4. **STRUCTURED DATA ACCESS:** Amazon RDS provides a relational database environment for accepted records.
5. **OPERATIONAL VISIBILITY:** Amazon SNS provides completion and alert notifications.
6. **BUSINESS INSIGHTS:** The dashboard makes sales information easier to interpret by presenting KPIs, trends, product performance, customer information, and geographic sales information.
7. **SCALABILITY:** The use of managed AWS services provides a foundation that can be expanded as data volumes and business requirements increase.

CHALLENGES ENCOUNTERED

Several challenges were encountered during the development of the project.

The first challenge was understanding how different AWS services work together to form a complete data pipeline.

Another challenge involved configuring IAM permissions so that AWS resources could access the resources they required.

Working with a real-world sales dataset also introduced data-quality challenges because not every record met the defined validation criteria.

Connecting serverless Lambda processing with Amazon RDS required an understanding of database connectivity, permissions, and AWS networking.

Developing the Sales KPI Dashboard also required transforming transactional data into meaningful business metrics.

Finally, organizing the project into a professional GitHub repository and documenting the architecture, implementation, results, and lessons learned was an important part of completing the project.

LESSONS LEARNED

The project provided practical experience in cloud data engineering and AWS architecture.

Key lessons learned include:

- Data quality is an essential part of a data pipeline.
- A data pipeline is not simply about moving data; data must be validated before it is trusted for analysis.
- AWS services should be selected according to the specific problem they solve.
- Serverless computing can simplify data-processing workloads.
- Separating rejected records improves traceability and makes data-quality issues easier to investigate.
- IAM permissions are critical when multiple AWS services communicate.
- Relational databases such as Amazon RDS provide structured storage for processed data.
- Notifications improve operational visibility.
- Data visualization helps transform technical data processing into business value.
- Good documentation is an important part of a data engineering project.

FUTURE ENHANCEMENTS

The pipeline can be extended in several ways.

Future improvements could include:

- More advanced data-quality validation.
- Schema validation and data-type checking.
- Duplicate-record detection.
- CloudWatch monitoring and alarms.
- More detailed SNS alert conditions.
- Interactive dashboard filters.
- Direct connection between the dashboard and the database.
- Amazon QuickSight for managed business intelligence.
- AWS Glue and Amazon Athena for larger-scale analytical workloads.
- Additional event-driven automation.

These improvements would make the solution more robust and suitable for larger production workloads.

CONCLUSION

The AWS Sales Data Pipeline demonstrates how AWS services can be combined to build a practical cloud-based data engineering solution. The pipeline processed 50,000 sales records and separated the records into valid and rejected datasets using defined validation rules. A total of 30,315 records were accepted, while 19,685 records were rejected. Accepted records were stored in Amazon RDS, while rejected records were retained separately in Amazon S3. Amazon SNS was used to provide pipeline completion and alert notifications. The processed data was also transformed into a Sales KPI Dashboard, providing business-focused insights into revenue, orders, products, customers, countries, and sales trends.

Overall, the project demonstrates practical knowledge of AWS cloud architecture, Amazon S3, AWS Lambda, Amazon RDS, Amazon SNS, AWS IAM, data validation, data pipeline design, visualization, GitHub deployment, and technical documentation.

PROJECT LINKS

GitHub Repository: <https://github.com/faith-ogundusi/aws-sales-data-pipeline>

Live Dashboard: <https://faith-ogundusi.github.io/aws-sales-data-pipeline/>